Open Data and ETLs

Data Sources, APIs, Extract, Transform, and Load

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-III





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Outline

1. Data Sources
2. Extract, Transform and Load





Outline

1 Data Sources

2 Extract, Transform and Load





Data Sources and Formats

Data Definition Framework

Data Source	Structured	Unstructured
Internal	**Human-Generated** • Survey ratings • Aptitude testing **Machine-Generated** • Web metrics from Web logs • Product purchase from sales Records • Process control measures	**Human-Generated** • Emails, letters, text messages • Audio transcripts • Customer comments • Voicemails • Corporate video/communications • Pictures, illustrations • Employee reviews
External	**Human-Generated** • Number of Retweets, Facebook likes, Google Plus +1s • Ratings on Yelp • Patient ratings ratings **Machine-Generated** • GPS for tweets • Time of tweet/updates/postings	**Human-Generated** • Content of social media updates • Comments in online forums • Comments on Yelp • Video reviews • Pinterest images • Surveillance video

Copyright 2014 Business Over Broadway

OpenData: Fundamentals

What is OpenData?

OpenData is **freely accessible**, **machine-readable**, and **openly licensed** data that anyone can use, modify, and redistribute without legal or technical restrictions.

- **Three Pillars**: accessibility (*free access*), reusability (*proper licensing*), and completeness (*full datasets, not summaries*).
- **Data Sources**: governments (`data.gov.co`), research institutions, international organizations (`UN`, `World Bank`), and scientific agencies (`NASA`).
- **Why OpenData Matters**: enables transparency, drives innovation, supports evidence-based policy, and democratizes access to information.





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

OpenData: Fundamentals

What is OpenData?

OpenData is **freely accessible**, **machine-readable**, and **openly licensed** data that anyone can use, modify, and redistribute without legal or technical restrictions.

- **Three Pillars**: accessibility (*free access*), reusability (*proper licensing*), and completeness (*full datasets not summaries*).
- **Data Sources**: governments (`data.gov.co`), research institutions, international organizations (`UN`, `World Bank`), and scientific agencies (`NASA`).
- **Why OpenData Matters**: enables transparency, drives innovation, supports evidence-based policy, and democratizes access to information.





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

OpenData: Fundamentals

What is OpenData?

OpenData is **freely accessible**, **machine-readable**, and **openly licensed** data that anyone can use, modify, and redistribute without legal or technical restrictions.

- **Three Pillars**: accessibility (*free access*), reusability (*proper licensing*), and completeness (*full datasets, not summaries*).
- **Data Sources**: governments (`data.gov.co`), research institutions, international organizations (`UN`, `World Bank`), and scientific agencies (`NASA`).
- **Why OpenData Matters**: enables transparency, drives innovation, supports evidence-based policy, and democratizes access to information.





OpenData: Fundamentals

What is OpenData?

OpenData is **freely accessible**, **machine-readable**, and **openly licensed** data that anyone can use, modify, and redistribute without legal or technical restrictions.

- **Three Pillars**: accessibility (*free access*), reusability (*proper licensing*), and completeness (*full datasets, not summaries*).
- **Data Sources**: governments (`data.gov.co`), research institutions, international organizations (`UN, World Bank`), and scientific agencies (`NASA`).
- **Why OpenData Matters**: enables transparency, drives innovation, supports evidence-based policy, and democratizes access to information.

CONPES 3920





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

OpenData: Standards and Licensing

Common OpenData Licenses:

- **CC0 (Creative Commons Zero)**: no restrictions, full public domain — most permissive.
- **CC-BY (Attribution)**: use freely, but must credit the original source.
- **CC-BY-SA (Share-Alike)**: credit required, and all derivatives must use same license.
- **ODBL (Open Data Commons By License)**: designed for databases; strict share-alike requirements for derived works.
- **Government-Specific**: each country/agency defines their own terms (e.g., Colombian datos.gov.co uses CC-BY 2.5 CO).

Best Practice

Always check the license before using open data. **Attribution** and **share-alike** requirements can impact your commercial use!



OpenData: Standards and Licensing

Common OpenData Licenses:

- **CC0 (Creative Commons Zero)**: no restrictions, full public domain — most permissive.
- **CC-BY (Attribution)**: use freely, but must credit the original source.
- **CC-BY-SA (Share-Alike)**: credit required, and all derivatives must use same license.
- **ODBL (Open Data Commons By License)**: designed for databases; strict share-alike requirements for derived works.
- **Government-Specific**: each country/agency defines their own terms (e.g., Colombian datos.gov.co uses CC-BY 2.5 CO).

Best Practice

Always check the license before using open data. **Attribution** and **share-alike** requirements can impact your commercial use!





OpenData: Standards and Licensing

Common OpenData Licenses:

- **CC0 (Creative Commons Zero)**: no restrictions, full public domain — most permissive.
- **CC-BY (Attribution)**: use freely, but must credit the original source.
- **CC-BY-SA (Share-Alike)**: credit required, and all derivatives must use same license.
- **ODBL (Open Data Commons By License)**: designed for databases; strict share-alike requirements for derived works.
- **Government-Specific**: each country/agency defines their own terms (e.g., Colombian `datos.gov.co` uses CC-BY 2.5 CO).

Best Practice

Always check the license before using open data. **Attribution** and **share-alike** requirements can impact your commercial use!



OpenData: Standards and Licensing

Common OpenData Licenses:

- **CC0 (Creative Commons Zero)**: no restrictions, full public domain — most permissive.
- **CC-BY (Attribution)**: use freely, but must credit the original source.
- **CC-BY-SA (Share-Alike)**: credit required, and all derivatives must use same license.
- **ODBL (Open Data Commons By License)**: designed for databases; strict share-alike requirements for derived works.
- **Government-Specific**: each country/agency defines their own terms (e.g., Colombian datos.gov.co uses CC-BY 2.5 CO).

Best Practice

Always check the license before using open data. **Attribution** and **share-alike** requirements can impact your commercial use!



OpenData: Standards and Licensing

Common OpenData Licenses:

- **CC0 (Creative Commons Zero)**: no restrictions, full public domain — most permissive.
- **CC-BY (Attribution)**: use freely, but must credit the original source.
- **CC-BY-SA (Share-Alike)**: credit required, and all derivatives must use same license.
- **ODBL (Open Data Commons By License)**: designed for databases; strict share-alike requirements for derived works.
- **Government-Specific**: each country/agency defines their own terms (e.g., Colombian `datos.gov.co` uses CC-BY 2.5 CO).

Best Practice

Always check the license before using open data. **Attribution** and **share-alike** requirements can impact your commercial use!



OpenData: Standards and Licensing

Common OpenData Licenses:

- **CC0 (Creative Commons Zero)**: no restrictions, full public domain — most permissive.
- **CC-BY (Attribution)**: use freely, but must credit the original source.
- **CC-BY-SA (Share-Alike)**: credit required, and all derivatives must use same license.
- **ODBL (Open Data Commons By License)**: designed for databases; strict share-alike requirements for derived works.
- **Government-Specific**: each country/agency defines their own terms (e.g., Colombian `datos.gov.co` uses CC-BY 2.5 CO).

Best Practice

Always check the license before using open data. **Attribution** and **share-alike** requirements can impact your commercial use!



API Fundamentals and REST

Application Programming Interface (API)

An API is a **contract** between systems defining how data is requested, validated, and delivered, typically via *HTTP* using *REST* architecture and *JSON* payloads.

- **REST API**: uses HTTP verbs (`GET`, `POST`, `PUT`, `DELETE`) and **URL endpoints** to expose resources as nouns.
- **Request Format**: `GET /api/v1/data?filter=value&limit=100 HTTP/1.1`
- **Response Format**: JSON with data, metadata, and error details.
- **Versioning**: APIs use versions (`/v1`, `/v2`) to maintain backward compatibility.





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

API Fundamentals and REST

Application Programming Interface (API)

An API is a **contract** between systems defining how data is requested, validated, and delivered, typically via *HTTP* using *REST* architecture and *JSON* payloads.

- **REST API**: uses HTTP verbs (`GET`, `POST`, `PUT`, `DELETE`) and **URL endpoints** to expose resources as nouns.
- **Request Format**: `GET /api/v1/data?filter=value&limit=100 HTTP/1.1`
- **Response Format**: JSON with data, metadata, and error details.
- **Versioning**: APIs use versions (`/v1`, `/v2`) to maintain backward compatibility.

request

Create / POST
Retrieve / GET
Update / PUT / PATCH
Delete / DELETE





API Fundamentals and REST

Application Programming Interface (API)

An API is a **contract** between systems defining how data is requested, validated, and delivered, typically via *HTTP* using *REST* architecture and *JSON* payloads.

- **REST API**: uses HTTP verbs (`GET`, `POST`, `PUT`, `DELETE`) and **URL endpoints** to expose resources as nouns.
- **Request Format** `GET /api/v1/data?filter=value&limit=100 HTTP/1.1`
- **Response Format**: JSON with data, metadata, and error details.
- **Versioning**: APIs use versions (`/v1`, `/v2`) to maintain backward compatibility.



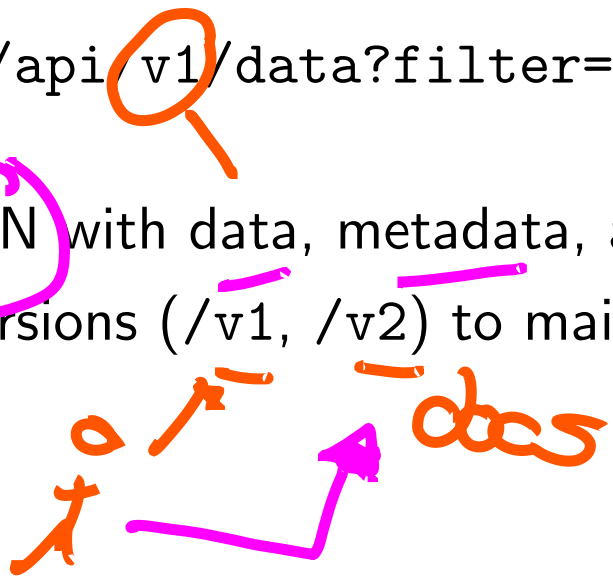


API Fundamentals and REST

Application Programming Interface (API)

An API is a **contract** between systems defining how data is requested, validated, and delivered, typically via *HTTP* using *REST* architecture and *JSON* payloads.

- **REST API**: uses HTTP verbs (`GET`, `POST`, `PUT`, `DELETE`) and **URL endpoints** to expose resources as nouns.
- **Request Format**: `GET /api/v1/data?filter=value&limit=100 HTTP/1.1`
- **Response Format**: JSON with data, metadata, and error details.
- **Versioning**: APIs use versions (`/v1`, `/v2`) to maintain backward compatibility.







API Authentication and Security

Authentication Methods

- **API Keys**: simple bearer tokens in HTTP headers.
- **OAuth 2.0**: industry standard for delegated access.
- **JWT (JSON Web Tokens)**: stateless token-based auth.
- **mTLS**: mutual certificate authentication.

Security Checklist

- Rotate keys every 90 days.
- Store credentials in `.env` files, **never** in source code.
- Use **HTTPS only** for production API calls.
- Validate **SSL** certificates to prevent MITM attacks.
- Rate-limit your own requests to avoid bans.





API Authentication and Security

Authentication Methods

- **API Keys**: simple bearer tokens in HTTP headers.
- **OAuth 2.0**: industry standard for delegated access.
- **JWT (JSON Web Tokens)**: stateless token-based auth.
- **mTLS**: mutual certificate authentication.

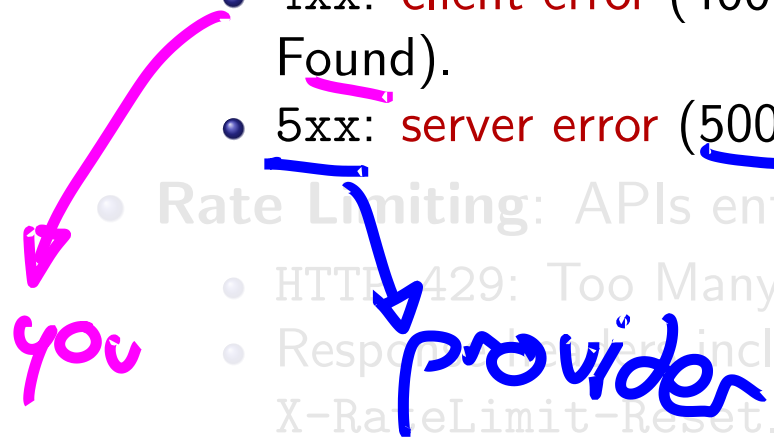
Security Checklist

- Rotate keys every 90 days.
- Store credentials in `.env` files, **never** in source code.
- Use **HTTPS only** for production API calls.
- **Validate SSL** certificates to prevent MITM attacks.
- Rate-limit your own requests to avoid bans.



Handling API Errors and Rate Limits

- **HTTP Status Codes**:
  - 2xx: success (200 OK, 201 Created).
  - 4xx: client error (400 Bad Request, 401 Unauthorized, 404 Not Found).
  - 5xx: server error (500 Internal Server Error, 503 Service Unavailable).
- **Rate Limiting**: APIs enforce request quotas (e.g., 30 requests/hour)
  - HTTP 429: Too Many Requests — your app exceeded the limit.
  - Response headers include `X-RateLimit-Remaining` and `X-RateLimit-Reset`.
- **Resilient Retry Strategy**:
  - Exponential backoff: wait 2^n seconds before retry (n = attempt count).
  - Circuit breaker: stop retrying after N consecutive failures.
  - Idempotency keys: use request IDs to safely replay failed requests.







UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Handling API Errors and Rate Limits

- **HTTP Status Codes**:
  - `2xx`: success (200 OK, 201 Created).
  - `4xx`: client error (400 Bad Request, 401 Unauthorized, 404 Not Found).
  - `5xx`: server error (500 Internal Server Error, 503 Service Unavailable).
- **Rate Limiting**: APIs enforce request quotas (e.g., 30 requests/hour)
  - `HTTP 429`: Too Many Requests — your app exceeded the limit.
  - Response headers include `X-RateLimit-Remaining` and `X-RateLimit-Reset`.
- **Resilient Retry Strategy**:
  - Exponential backoff: wait 2^n seconds before retry (n = attempt count).
  - Circuit breaker: stop retrying after N consecutive failures.
  - Idempotency keys: use request IDs to safely replay failed requests.





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Handling API Errors and Rate Limits

- **HTTP Status Codes**:
  - `2xx`: success (200 OK, 201 Created).
  - `4xx`: client error (400 Bad Request, 401 Unauthorized, 404 Not Found).
  - `5xx`: server error (500 Internal Server Error, 503 Service Unavailable).
- **Rate Limiting**: APIs enforce request quotas (e.g., 30 requests/hour)
  - `HTTP 429`: Too Many Requests — your app exceeded the limit.
  - Response headers include `X-RateLimit-Remaining` and `X-RateLimit-Reset`.
- **Resilient Retry Strategy**:
  - Exponential backoff: wait 2^n seconds before retry (n = attempt count).
  - Circuit breaker: stop retrying after N consecutive failures.
  - Idempotency keys: use request IDs to safely replay failed requests.



Outline

1 Data Sources

2 Extract, Transform and Load

ETL



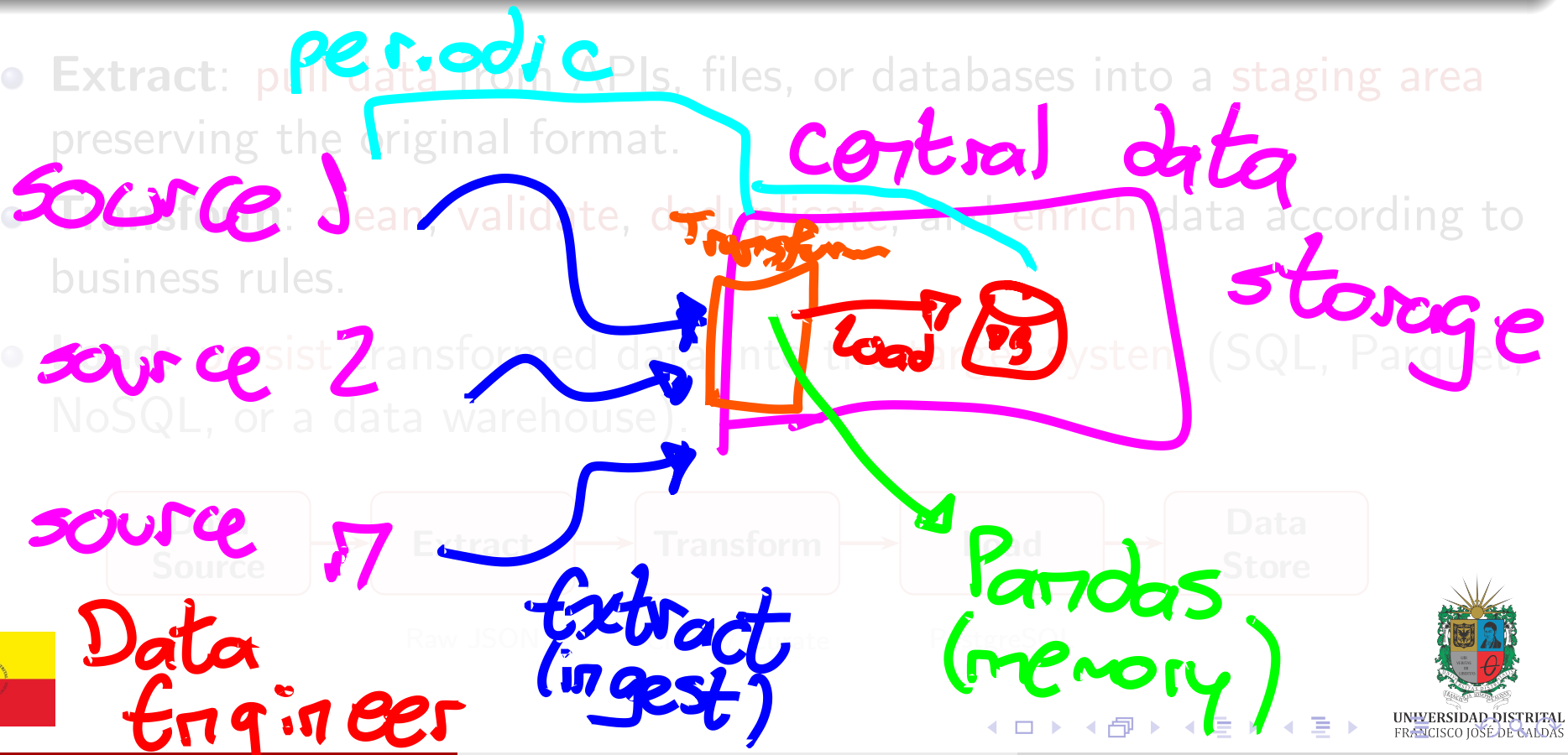


UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Extract-Transform-Load (ETL) Process

ETL

ETL is a **data integration pattern** that extracts data from sources, transforms it to meet quality and business rules, and loads it into a destination system ready for *analysis*.



Extract-Transform-Load (ETL) Process

ETL

ETL is a **data integration pattern** that extracts data from sources, transforms it to meet quality and business rules, and loads it into a destination system ready for *analysis*.

- **Extract**: pull data from APIs, files, or databases into a staging area preserving the original format.
- **Transform**: clean, validate, deduplicate, and enrich data according to business rules.
- **Load**: persist transformed data into the target system (SQL, Parquet, NoSQL, or a data warehouse).

avoid loss





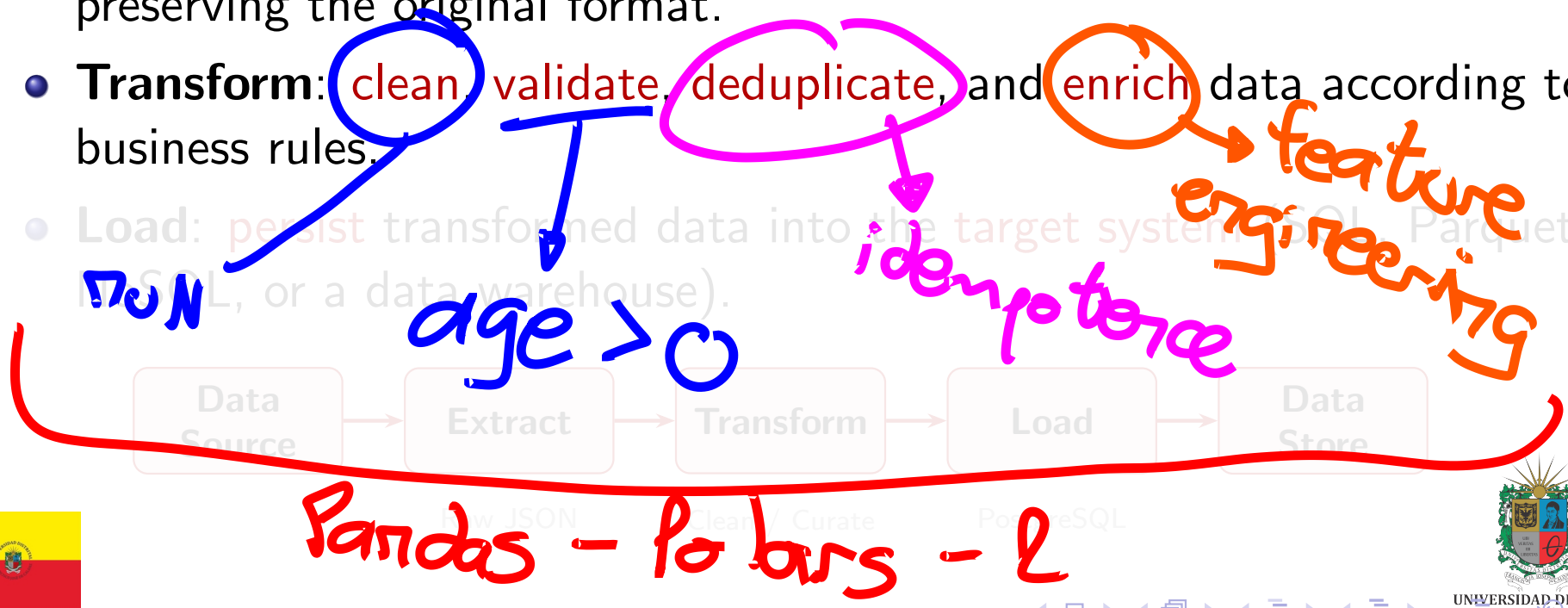
UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Extract-Transform-Load (ETL) Process

ETL

ETL is a **data integration pattern** that extracts data from sources, transforms it to meet quality and business rules, and loads it into a destination system ready for *analysis*.

- **Extract**: pull data from APIs, files, or databases into a staging area preserving the original format.
- **Transform**: clean, validate, deduplicate, and enrich data according to business rules.
- **Load**: persist transformed data into the target system (PostgreSQL, Parquet, NoSQL, or a data warehouse).

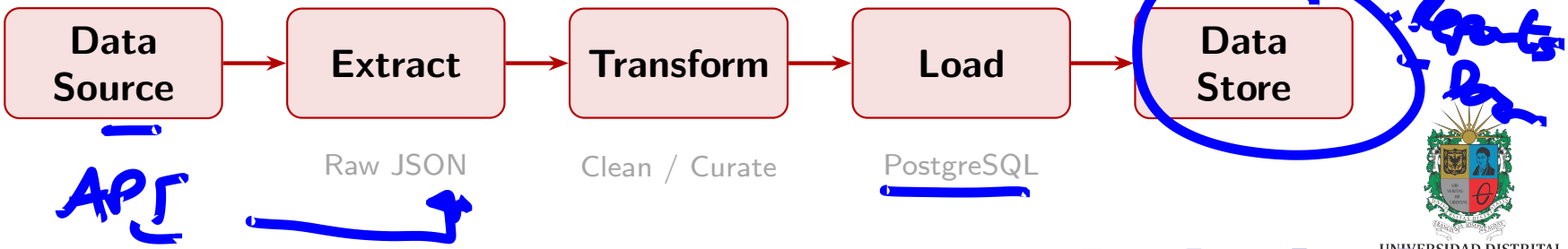


Extract-Transform-Load (ETL) Process

ETL

ETL is a **data integration pattern** that extracts data from sources, transforms it to meet quality and business rules, and loads it into a destination system ready for *analysis*.

- **Extract**: pull data from APIs, files, or databases into a staging area preserving the original format.
- **Transform**: clean, validate, deduplicate, and enrich data according to business rules.
- **Load**: persist transformed data into the target system (SQL, Parquet, NoSQL, or a data warehouse).



ETL vs ELT: Modern Data Pipelines

Traditional ETL

Extract → Transform → Load

- Transform before load.
- Schema enforced at write time.
- Best for structured data.
- Higher upfront transformation cost.

Modern ELT

Extract → Load → Transform

- Load raw data first (Bronze layer).
- Schema applied at query time.
- Flexible for all data types.
- Defer expensive transforms.

Our approach follows **ELT**: raw JSON → PostgreSQL → Parquet summaries.





ETL vs ELT: Modern Data Pipelines

Traditional ETL

Extract → **Transform** → **Load**

- Transform before load.
- Schema enforced at write time.
- Best for structured data.
- Higher upfront transformation cost.

Modern ELT

Extract → **Load** → **Transform**

- Load raw data first (Bronze layer).
- Schema applied at query time.
- Flexible for all data types.
- Defer expensive transforms.

Our approach follows **ELT**: raw JSON → PostgreSQL → Parquet summaries



ETL vs ELT: Modern Data Pipelines

Traditional ETL

Extract → Transform → Load

- Transform before load.
- Schema enforced at write time.
- Best for structured data.
- Higher upfront transformation cost.

Modern ELT

Extract → Load → Transform

- Load raw data first (Bronze layer).
- Schema applied at query time.
- Flexible for all data types.
- Defer expensive transforms.

Our approach follows **ELT**: raw `JSON` → `PostgreSQL` → `Parquet` summaries.



Data Quality Framework

Garbage In, Garbage Out

Five Dimensions of Data Quality

High-quality data requires multiple quality attributes to be satisfied **before** downstream analysis and decision-making.

- **Completeness**: $\geq 95\%$ of records contain all required fields; missing values documented and handled.
- **Accuracy**: data reflects reality, validated via *checksums*, *cross-checks*, *domain rules*.
- **Consistency**: same entity has uniform representation across sources (date formats, units, terminology).
- **Timeliness**: data is current and up-to-date per service level agreements (SLAs).
- **Uniqueness**: no duplicates; detected via exact match, fuzzy matching, composite keys.

imputing





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Data Quality Framework

Five Dimensions of Data Quality

High-quality data requires multiple quality attributes to be satisfied **before** downstream analysis and decision-making.

- **Completeness**: $\geq 95\%$ of records contain all required fields; missing values documented and handled.
- **Accuracy**: data reflects reality; validated via *checksums*, *cross-checks*, *domain rules*.
- **Consistency**: same entity has uniform representation across sources (date formats, units, terminology).
- **Timeliness**: data is current and up-to-date per service-level agreements (SLAs).
- **Uniqueness**: no duplicates; detected via exact match, fuzzy matching, composite keys.



Data Quality Framework

Five Dimensions of Data Quality

High-quality data requires multiple quality attributes to be satisfied **before** downstream analysis and decision-making.

- **Completeness**: $\geq 95\%$ of records contain all required fields; missing values documented and handled.
- **Accuracy**: data reflects reality; validated via *checksums*, *cross-checks*, *domain rules*.
- **Consistency**: same entity has uniform representation across sources (date formats, units, terminology).
- **Timeliness**: data is current and up-to-date per service level agreements (SLAs).
- **Uniqueness**: no duplicates; detected via exact match, fuzzy matching, composite keys.



Data Quality Framework

Five Dimensions of Data Quality

High-quality data requires multiple quality attributes to be satisfied **before** downstream analysis and decision-making.

- **Completeness**: $\geq 95\%$ of records contain all required fields; missing values documented and handled.
- **Accuracy**: data reflects reality; validated via *checksums*, *cross-checks*, *domain rules*.
- **Consistency**: same entity has uniform representation across sources (date formats, units, terminology).
- **Timeliness**: data is current and up-to-date per service level agreements (SLAs).
- **Uniqueness**: no duplicates; detected via exact match, fuzzy matching, composite keys.





Data Quality Framework

Five Dimensions of Data Quality

High-quality data requires multiple quality attributes to be satisfied **before** downstream analysis and decision-making.

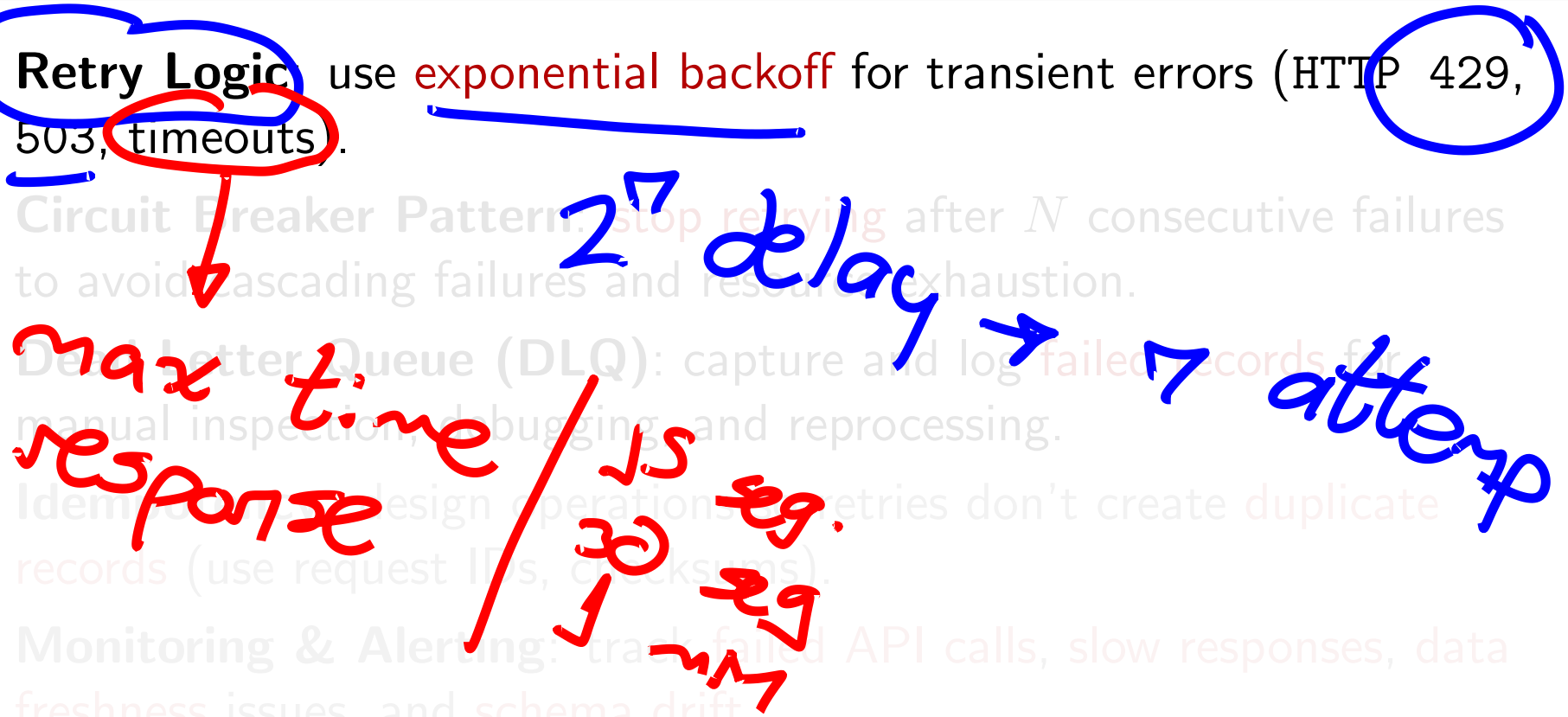
- **Completeness**: $\geq 95\%$ of records contain all required fields; missing values documented and handled.
- **Accuracy**: data reflects reality; validated via *checksums*, *cross-checks*, *domain rules*.
- **Consistency**: same entity has uniform representation across sources (date formats, units, terminology).
- **Timeliness**: data is current and up-to-date per service level agreements (SLAs).
- **Uniqueness**: no duplicates; detected via exact match, fuzzy matching, composite keys.





Error Handling and Resilience

- **Retry Logic**: use exponential backoff for transient errors (`HTTP 429`, 503, timeouts).
- **Circuit Breaker Pattern**: stop retrying after N consecutive failures to avoid cascading failures and resource exhaustion.
- **Dead Letter Queue (DLQ)**: capture and log failed records for manual inspection, debugging, and reprocessing.
- **Idempotency**: design operations so retries don't create duplicate records (use request IDs, checksums).
- **Monitoring & Alerting**: track failed API calls, slow responses, data freshness issues, and schema drift.



Error Handling and Resilience

- **Retry Logic**: use exponential backoff for transient errors (`HTTP 429`, 503, timeouts).
- **Circuit Breaker Pattern**: stop retrying after N consecutive failures to avoid cascading failures and resource exhaustion.
- **Dead Letter Queue (DLQ)**: capture and log failed records for manual inspection, debugging, and reprocessing.
- **Idempotency**: design operations so retries don't create duplicate records (use request IDs, checksums).
- **Monitoring & Alerting**: track failed API calls, slow responses, data freshness issues, and schema drift.





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Error Handling and Resilience

- **Retry Logic**: use exponential backoff for transient errors (`HTTP 429`, 503, timeouts).
- **Circuit Breaker Pattern**: stop retrying after N consecutive failures to avoid cascading failures and resource exhaustion.
- **Dead Letter Queue (DLQ)**: capture and log failed records for manual inspection, debugging, and reprocessing.
- **Idempotency**: design operations so retries don't create duplicate records (e.g., request IDs, checksums).
- **Monitoring & Alerting**: track failed API calls, slow responses, data freshness issues, and schema drift.

cloud engineer
platform engineer





Error Handling and Resilience

- **Retry Logic**: use exponential backoff for transient errors (`HTTP 429`, 503, timeouts).
- **Circuit Breaker Pattern**: stop retrying after N consecutive failures to avoid cascading failures and resource exhaustion.
- **Dead Letter Queue (DLQ)**: capture and log failed records for manual inspection, debugging, and reprocessing.
- **Idempotency**: design operations so retries don't create duplicate records (use request IDs, checksums).
- **Monitoring & Alerting**: track failed API calls, slow responses, data freshness issues, and schema drift.

↳ context + metadata





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

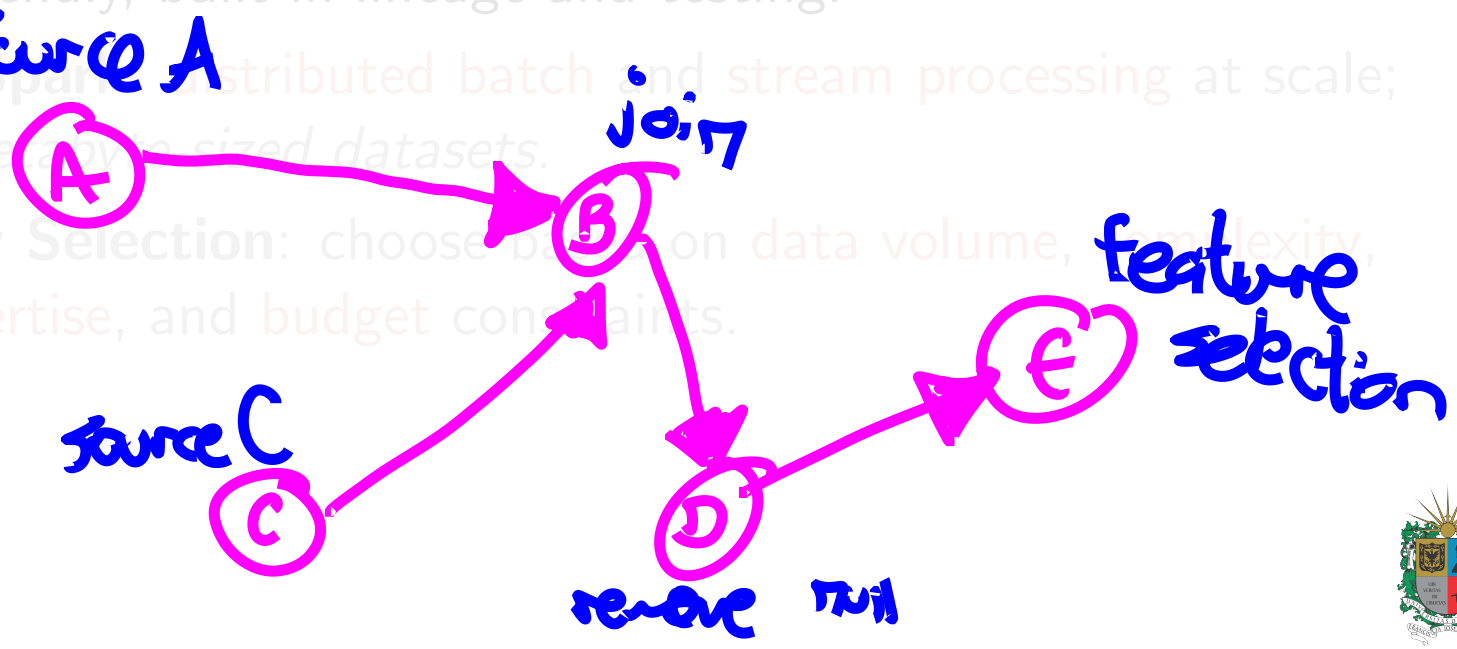
Error Handling and Resilience

- **Retry Logic**: use exponential backoff for transient errors (`HTTP 429`, 503, timeouts).
- **Circuit Breaker Pattern**: stop retrying after N consecutive failures to avoid cascading failures and resource exhaustion.
- **Dead Letter Queue (DLQ)**: capture and log failed records for manual inspection, debugging, and reprocessing.
- **Idempotency**: design operations so retries don't create duplicate records (use request IDs, checksums).
- **Monitoring & Alerting**: track failed API calls, slow responses, data freshness issues, and schema drift.



ETL Orchestration Tools

- **Apache Airflow**: `Python`-based DAGs, rich web UI, *XCom* for inter-task communication. *Best for:* complex workflows, conditional logic.
- **Prefect**: modern, async-first orchestrator with better error handling and retry logic out of the box.
- **dbt (data build tool)**: SQL-focused transformations, version control-friendly, built-in lineage and testing.
- **Apache Spark**: distributed batch and stream processing at scale; handles *petabyte-sized datasets*.
- **Workflow Selection**: choose based on data volume, complexity, team expertise, and budget constraints.



ETL Orchestration Tools

- **Apache Airflow**: `Python`-based DAGs, rich web UI, *XCom* for inter-task communication. *Best for:* complex workflows, conditional logic.
- **Prefect**: modern, async-first orchestrator with better error handling and retry logic out of the box.
- **dbt (data build tool)**: `SQL`-focused transformations, version control-friendly, built-in lineage and testing.
- **Apache Spark**: distributed batch and stream processing at scale; handles *petabyte-sized datasets*.
- **Workflow Selection**: choose based on data volume, complexity, team expertise, and budget constraints.





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

ETL Orchestration Tools

- **Apache Airflow**: `Python`-based DAGs, rich web UI, *XCom* for inter-task communication. *Best for:* complex workflows, conditional logic.
- **Prefect**: modern, async-first orchestrator with better error handling and retry logic out of the box.
- **dbt (data build tool)**: `SQL`-focused transformations, version control-friendly, built-in lineage and testing.
- **Apache Spark**: distributed batch and stream processing at scale; handles *petabyte-sized datasets*.
- **Workflow Selection**: choice based on data volume, complexity, team expertise, and budget constraints.

DataBricks





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

ETL Orchestration Tools

- **Apache Airflow**: `Python`-based DAGs, rich web UI, *XCom* for inter-task communication. *Best for:* complex workflows, conditional logic.
- **Prefect**: modern, async-first orchestrator with better error handling and retry logic out of the box.
- **dbt (data build tool)**: `SQL`-focused transformations, version control-friendly, built-in lineage and testing.
- **Apache Spark**: distributed batch and stream processing at scale; handles *petabyte-sized datasets*.
- **Workflow Selection**: Choose based on data volume, complexity, team expertise, and budget constraints.



ETL Orchestration Tools

- **Apache Airflow**: `Python`-based DAGs, rich web UI, *XCom* for inter-task communication. *Best for:* complex workflows, conditional logic.
- **Prefect**: modern, async-first orchestrator with better error handling and retry logic out of the box.
- **dbt (data build tool)**: `SQL`-focused transformations, version control-friendly, built-in lineage and testing.
- **Apache Spark**: distributed batch and stream processing at scale; handles *petabyte-sized datasets*.
- **Workflow Selection**: choose based on data volume, complexity, team expertise, and budget constraints.

↳ Snowflake





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

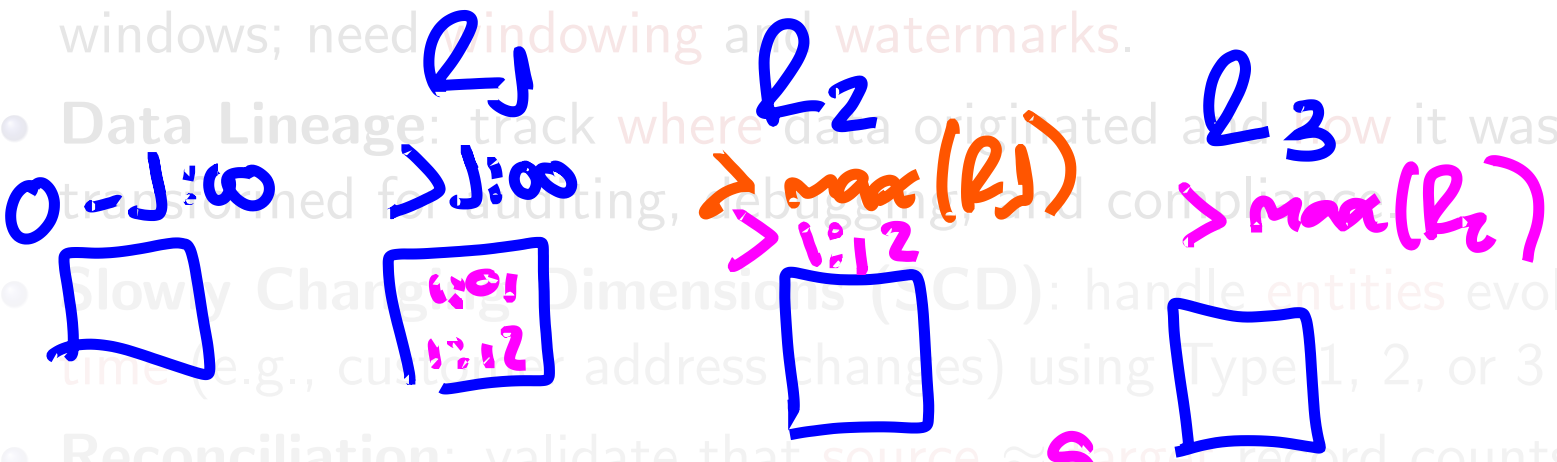
ETL Monitoring and Observability

Key Metrics to Monitor

- **DAG Health**: task success rate, average execution time, failures by type.
- **Data Freshness**: time elapsed since last successful run; alert if stale.
- **Row Count Validation**: alert if record count $\pm 20\%$ from baseline.
- **Schema Drift**: detect unexpected columns, type changes, or missing fields.
- **Latency**: end-to-end pipeline duration; alert if exceeds SLA.

Real-World ETL Complexity

- **Incremental Loads**: only ingest new/changed records since last run using timestamps or *change data capture (CDC)*.
- **Late-Arriving Data**: records may arrive out of order or after cutoff windows; need windowing and watermarks.
- **Data Lineage**: track where data originated and how it was transformed for auditing, debugging, and compliance.
- **Slowly Changing Dimensions (SCD)**: handle entities evolving over time (e.g., customer address changes) using Type 1, 2, or 3 strategies.
- **Reconciliation**: validate that source ≈ target record counts after each load.



ETL → each 5 minutes
15 minutes
30 minutes
24 hours

Real-World ETL Complexity

- **Incremental Loads**: only ingest new/changed records since last run using timestamps or *change data capture (CDC)*.
- **Late-Arriving Data**: records may arrive out of order or after cutoff windows; need windowing and watermarks.
- **Data Lineage**: track where data originated and how it was transformed for auditing, debugging, and compliance.
- **Slowly Changing Dimensions (SCD)**: handle entities evolving over time (e.g., customer address changes) using Type 1, 2, or 3 strategies.
- **Reconciliation**: validate that source $\approx$ target record counts after each load.





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Real-World ETL Complexity

- **Incremental Loads**: only ingest new/changed records since last run using timestamps or *change data capture (CDC)*.
- **Late-Arriving Data**: records may arrive out of order or after cutoff windows; need windowing and watermarks.
- **Data Lineage**: track where data originated and how it was transformed for auditing, debugging, and compliance.
- **Slowly Changing Dimensions (SCD)**: handle entities evolving over time (e.g., customer address changes) using Type 1, 2, or 3 strategies.
- **Reconciliation**: verify that source ≈ target record counts after each load.

Open Metadata | versions dataset

cloud

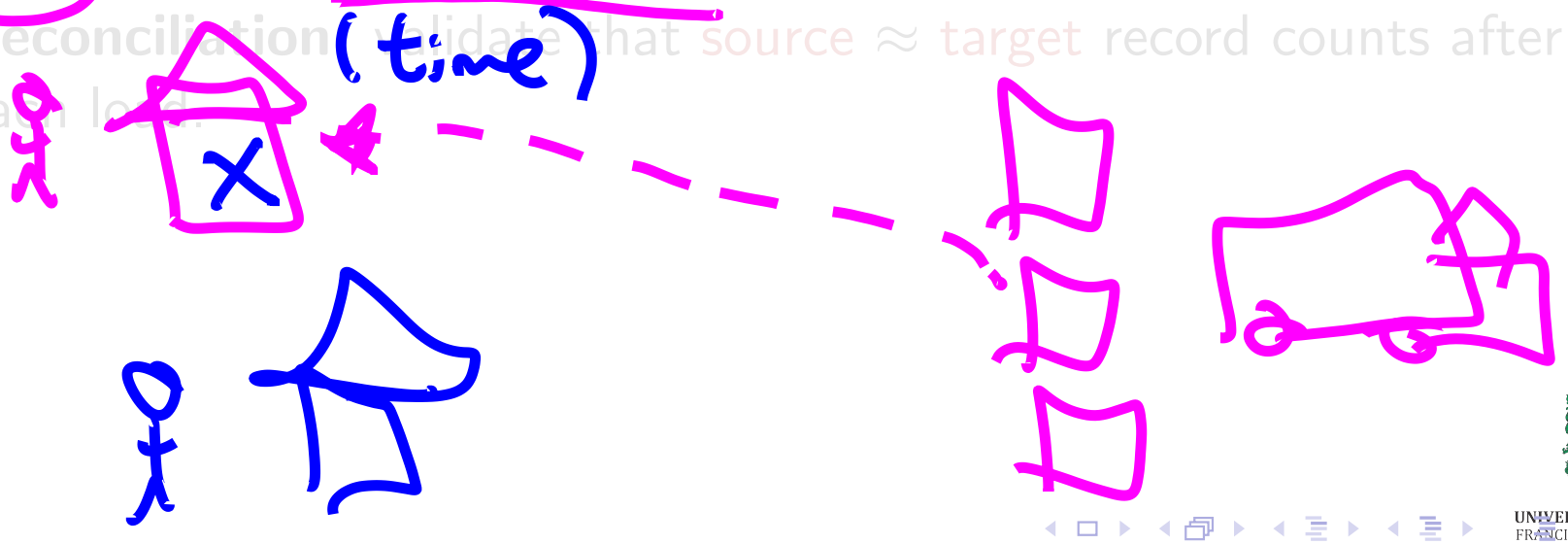




UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS

Real-World ETL Complexity

- **Incremental Loads**: only ingest new/changed records since last run using timestamps or *change data capture (CDC)*.
- **Late-Arriving Data**: records may arrive out of order or after cutoff windows; need windowing and watermarks.
- **Data Lineage**: track where data originated and how it was transformed for auditing, debugging, and compliance.
- **Slowly Changing Dimensions (SCD)**: handle entities evolving over time (e.g., customer address changes) using Type 1, 2, or 3 strategies.
- **Reconciliation**: validate that source ≈ target record counts after each load.



Real-World ETL Complexity

- **Incremental Loads**: only ingest new/changed records since last run using timestamps or *change data capture (CDC)*.
- **Late-Arriving Data**: records may arrive out of order or after cutoff windows; need windowing and watermarks.
- **Data Lineage**: track where data originated and how it was transformed for auditing, debugging, and compliance.
- **Slowly Changing Dimensions (SCD)**: handle entities evolving over time (e.g., customer address changes) using Type 1, 2, or 3 strategies.
- **Reconciliation**: validate that source $\approx$ target record counts after each load.

→ automatic process





ETL Performance and Scaling

Optimization Strategies

- **Partitioning**: split large datasets by date, region, or category for parallel processing.
- **Columnar Storage**: use Parquet/ORC for 5-10x compression vs. CSV.
- **Incremental Processing**: avoid full table scans; only process changed data since last run.
- **Distributed Execution**: leverage parallelism across CPU cores and cluster nodes (Spark, Dask).





ETL Performance and Scaling

Optimization Strategies

- **Partitioning**: split large datasets by date, region, or category for parallel processing.
- **Columnar Storage**: use Parquet/ORC for 5-10x compression vs. CSV.
- **Incremental Processing**: avoid full table scans; only process changed data since last run.
- **Distributed Execution**: leverage parallelism across CPU cores and cluster nodes (Spark, Dask).







UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Outline

1 Data Sources

2 Extract, Transform and Load





Thanks!

Questions?

Repo: *https://github.com/EngAndres/ud-public/tree/main/courses/data-analysis-programming*





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS